snnbench Evaluation — What's Worth Extracting Into SNNs-auf-GPUs

Independent analysis of the snnbench workspace against the current state of SNNs-auf-GPUs

Scope: Independent analysis of `C:\Users\zuhai\Desktop\Projects\SNN\snnbench`, a separate workspace, against the current state of this repository (`SNNs-auf-GPUs`). Written to answer one question: does snnbench contain anything that would concretely improve this project's architecture or scientific validity, and if so, what exactly should be extracted versus left alone.

> **Headline verdict:** Most of snnbench is a self-contained research instrument built to answer *snnbench's own* questions (kernel-fusion speedup, T-scaling exponents, the "neuromorphic gap" between measured and theoretical energy) and isn't a fit to port wholesale — that instinct to "dismantle" most of it is correct. But one piece of it exposes a real, checkable gap in this project's own methodology: **this repo trains and compares four SNN frameworks side by side (`docs/results/`, `docs/framework_comparison_report.md`) without ever verifying they compute the same neuron model.** snnbench's entire design exists to prevent exactly that mistake, and applying even a small piece of it against this repo's current config surfaced a live, currently-present bug (below). That's the concrete return on this analysis, not just an abstract "good idea."

1. What snnbench actually is

snnbench is a controlled cross-framework SNN benchmark with six experiments, each mapped to a hypothesis with a pre-registered pass/fail rule (`analyze.py`'s `evaluate_hypotheses()`, hypotheses H1–H6):

EXPERIMENT	FILE	QUESTION	HYPOTHESIS
A — Equivalence	`equivalence.py`	Do the frameworks compute the same thing?	*(prerequisite for everything else)*
B — Runtime scaling	`microbench.py`	How does step time scale with T, width, depth, batch?	H1 (fusion ≥3× at large T), H2 (~linear in T)
C — Memory / OOM frontier	`microbench.py`	Where's the OOM wall?	H2
D — Accuracy	`train.py`	Does framework choice change reachable accuracy at matched everything-else?	H4 (no, within noise)

E — Real-time latency	`realtime.py`	Is the whole pipeline (not just the forward pass) fast enough for a deadline?	H6 (preprocessing dominates for small models)
F — Energy	`realtime.py`	Measured GPU energy vs. SynOps theory	H5 (GPU energy tracks dense work, not sparsity)

The organizing idea, stated explicitly in `equivalence.py`'s module docstring, is: *"This experiment runs FIRST, and everything else is conditional on it. If two frameworks do not compute the same function, comparing their runtimes is comparing different workloads and any speed ranking is meaningless."* Every other file assumes that check has already passed.

This is a fundamentally different project shape from ours. Our repo's four frameworks (`frameworks/snn_norse.py`, `snn_torch.py`, `snn_spikingjelly.py`, `snn_sinabs.py`) each get tuned to train well independently on a chosen dataset — the goal is "does this framework train this architecture successfully," not "are these frameworks numerically the same model." snnbench's goal is the opposite and stricter: hold the model mathematically fixed and vary only the framework, so that a runtime/memory/energy comparison is actually valid. Both are legitimate goals — but our `docs/results/README.md` and `docs/framework_comparison_report.md` *do* make snnbench-style comparative claims (accuracy, energy, latency, side by side across frameworks) without snnbench's prerequisite check. That mismatch is where the real, actionable finding lives (§4).

2. The neuron model — `neurons.py` vs. our `frameworks/snn_lif.py`

Both projects independently built a from-scratch LIF neuron in plain PyTorch. Comparing them side by side:

	SNNBENCH (NEURONS.PY)	SNNS-AUF-GPUS (FRAMEWORKS/SNN_LIF.PY)
Dynamics	`H[t] = beta·V[t-1] + I[t]`, documented explicitly as the *normative spec* other frameworks are checked against	Same recurrence, same hard reset — but never checked against anything
Surrogate gradient	ATan — chosen because it's the same functional form SpikingJelly's `ATan` and snnTorch's `surrogate.atan` use, making forward AND some gradient comparisons meaningful	Fast-sigmoid — a different functional family from what any of our other four frameworks use natively
Reset modes	Both hard (`zero`) and soft (`subtract`) reset, selectable	Hard reset only
Fused variant	`FusedLIFLayer` — same math, restructured so `torch.compile` can fuse the elementwise	Not present

	chain, so H1 (fusion speedup) is measured against *this exact model*	
Self-instrumentation	`RefLIFLayer.firing_rate` — spike count / element count, no second pass needed	Handled separately via `ActivityMonitor` — comparable capability, different mechanism
Validation primitive	`subthreshold_trace()` — closed-form membrane trace, checks the `beta` ↔decay mapping before attempting the harder spike-level comparison	Nothing analogous exists

The two neuron cores are functionally close cousins — same discrete LIF recurrence, same idea of a from-scratch reference implementation. The meaningful gap isn't the neuron itself, it's everything snnbench wraps around it to make it *checkable*: the ATan choice exists specifically so cross-framework comparison is possible, and `subthreshold_trace` exists specifically to catch parameter-mapping mistakes cheaply, before running a full spiking forward pass. `SNN_LIF` has neither property — it's currently a fifth, free-floating implementation that isn't wired into `learning/main.py`'s `MODELS` dict at all (confirmed: only `norse` / `torch` / `sj` / `sinabs` are dispatchable; `SNN_LIF` only runs via its own `__main__` block).

3. `microbench.py` vs. our `calibrate_batch_size`

You singled out `microbench.py` as interesting, and it's the strongest engineering artifact in the repo. What it does that ours doesn't:

- **One-factor-at-a-time sweep** (`sweep_axis`) across T, width, depth, batch size, each varied independently around a base point — this repo has no equivalent scan; `calibrate_batch_size` is a single live probe that answers "what batch size fits *right now*," not "how does memory/time scale as any of these axes move."
- **The one deliberate 2-D grid** (`grid_T_batch`): T × batch_size specifically, because BPTT activation memory is `O(T × B × neurons)` and the two axes genuinely interact. This is exactly the memory story `docs/functions.md` already tells about `calibrate_batch_size` (N-MNIST 34×34 vs. N-Caltech101 240×180 OOM-ing at the same batch size) — snnbench's version generalizes it into a repeatable, swept characterization instead of a single measured point per dataset.
- **OOM is data, not an exception**: `measure_one` never raises — every failure becomes a row with `status="oom"` , so the OOM frontier itself is a first-class result. Our `measure_batch_vram` does the same in spirit but only for the single point it's currently probing.
- **Timing discipline**: CUDA events, discard warmup iterations, report median + IQR (not mean — GPU clocks drift with thermal/power state), crash-safe JSONL append per record.
- **Firing rate as a controlled variable**: `calibrate_firing_rate` bisects an init-weight gain so every point in a sweep starts at the *same* mean firing rate, specifically because "a silent

network is fast and useless, and a saturated one is slow and equally useless." This repo has no equivalent: our four frameworks' networks fire at whatever rate their initialization and per-framework threshold/beta happen to produce, uncontrolled.

4. The concrete finding: our cross-framework neuron parameters are not verified equivalent, and one is already documented-but-uncorrected

This is the part worth the most attention. Applying snnbench's premise — "check the parameter mapping before trusting a comparison" — against this repo's actual `configuration/SNN_module.yaml` surfaced this:

```
# threshold=0.5: SNNTorch amplifies input 20x (V_eq=20I), Norse/SpikingJelly do not (V_eq=I).
# threshold=1.0 kills Norse (dead neurons, confirmed EXP-004). 0.5 is a safe neutral starting
frameworks:
  snntorch:
    beta:       0.95
    threshold:  0.5
  norse:
    tau_mem_inv: 100.0
    threshold:   1.0        # <- this is the exact value the comment above says kills it
  spikingjelly:
    tau:         2.0
    threshold:   0.5
  sinabs:
    tau_mem:     20.0
    threshold:   0.5
```

The comment sitting directly above the `norse:` block states, in the same document, that `threshold=1.0` is known to kill Norse (dead neurons, "confirmed EXP-004") — and then `norse.threshold` is set to exactly `1.0`. This is either a live regression (someone reverted the fix, or the fix was never applied) or the comment is stale from an earlier iteration and no longer describes the current test. Either way, it's an internal contradiction sitting in the config file today. This is precisely the class of bug snnbench's `check_subthreshold` (A1) or even just `calibrate_firing_rate` would catch automatically and cheaply — both work by driving a single neuron layer and confirming it actually responds, before anyone spends a training run on it. **This value has not been changed** — flagging it here since it surfaced directly from this analysis.

Separately, and by design rather than by accident: `frameworks/snn_norse.py`'s own comment states *"Do NOT convert from SNNTorch beta: different coordinate systems... SNNTorch: V_eq = 20·I (20x amplification). Norse: V_eq = I (no amplification)."* — this repo already knows and documents that its frameworks are **not** parameterized to compute the same membrane dynamics. That's a defensible choice if the goal is "each framework trains this architecture well on its own terms." It becomes a problem the moment a document puts their accuracy/energy/latency

numbers in one table and lets a reader compare rows — that comparison is only meaningful if a reader is told, explicitly, which differences are model differences and which are implementation differences. snnbench's `_verdict()` function in `equivalence.py` does exactly this bookkeeping (`"DIFFERENT MODEL — do not compare runtimes without a caveat"` vs. `"forward-identical"` vs. `"gradients correlated, different surrogate"`) and is the cleanest piece of this whole codebase to lift directly.

5. Recommendation

Extract and adapt (highest value, lowest cost):

1. **The equivalence-check pattern (`equivalence.py`), scaled down.** Not the full A1/A2/A3 apparatus verbatim — a lightweight version that: (a) runs a `check_subthreshold`-style validation across all five of our neuron implementations (`norse`, `torch`, `sj`, `sinabs`, `SNN_LIF`) whenever `training.framework` or the `frameworks:` YAML block changes, and (b) fails loudly if a configured threshold/beta/tau combination produces a dead or saturated network — which would have caught the Norse `threshold=1.0` issue in §4 automatically, before a training run wastes GPU time on it.
2. `subthreshold_trace` **as a standalone sanity check.** Cheap (no spiking, no gradient, closed-form), and directly answers "did I convert this framework's native decay parameter correctly" — the single most common source of silent cross-framework mismatch per snnbench's own design notes.
3. **A generalized version of `calibrate_batch_size` into a small sweep**, reusing `microbench.py`'s `sweep_axis` / `grid_T_batch` pattern (not the file itself) to characterize the T×batch OOM frontier per dataset, rather than a single live-probed point.
4. `calibrate_firing_rate`**'s core idea** — even just *reporting* each framework's mean firing rate at a matched config, so results tables can flag when one framework's accuracy/energy numbers come from a near-dead or saturated network (this already happened once: `docs/results/README.md` documents Sinabs ending a run with zero output spikes, purely by observation after the fact — a firing-rate check up front would catch this before the run, not after).

Adapt with caution (real technique, needs framework-specific verification before trusting it):

5. The `beta` ↔framework-native-parameter conversion formulas in `config.py` are specific to snnbench's own reset/threshold conventions and would need re-deriving against this repo's actual per-framework setup (e.g., SNNTorch's 20× input amplification noted in `snn_norse.py`'s own comment) — don't copy the formulas literally, copy the *method* of deriving and stating them explicitly.

Leave alone / don't port (matches the instinct that most of the workspace is a small, self-contained evaluation):

6. `train.py`, `realtime.py`, `analyze.py`'s hypothesis-testing machinery (H1–H6), the CLI, and the preset system are snnbench's own research-question scaffolding. These answer snnbench's specific thesis questions, not this project's. Porting them would import a second, parallel experiment-tracking system rather than strengthening this one.
7. `data.py`'s `SyntheticEvents` generator is a reasonable idea but this repo already has five real event datasets wired through `DATASET_REGISTRY` and doesn't need a synthetic stand-in.

6. Suggested next step, if you want to act on this

Smallest useful slice: pull `neurons.py`'s `subthreshold_trace` and a trimmed version of `check_subthreshold` into a new small module (e.g. `frameworks/neuron_equivalence.py`), run it once across all five of this repo's neuron implementations at their *currently configured* `SNN_module.yaml` parameters, and see what it reports for Norse specifically. That single run would resolve the §4 finding directly — either the network really is dead at `threshold=1.0` (matching the comment) and needs the same fix EXP-004 presumably applied once already, or something else about the current setup makes it fine now and the comment is what's stale.